Bots and how to catch them:
A mathematical detector for romance–scam chatbots at the point of financial harm

STEM Games 2026
Mathematics arena × Erste

Team name and team members

Sažetak

We design and implement a mathematical detector for romance–scam chatbots, intended for deployment at the moment a bank customer initiates an outgoing transfer. A suspected conversation is mapped to a feature vector combining (i) token–level likelihood statistics under a reference language model (perplexity, burstiness, and a curvature score in the spirit of DetectGPT), (ii) classical stylometric statistics (type–token ratio, Yule's K, deviation from Zipf's law), (iii) conversational–dynamics statistics (length asymmetry, sentiment trajectory analyzed via a CUSUM change–point test, and topic–shift measured by Kullback–Leibler divergence), (iv) a hidden Markov model whose hidden states encode the canonical stages of the romance–scam playbook, and (v) semantic similarity to a corpus of confirmed scam messages obtained via a sentence–transformer. Features are fused with ℓ_2–regularized logistic regression. We report cross–validated ROC AUC, precision–recall curves, and a five–family feature–ablation analysis. Strengths, failure modes (paraphrase attacks, mixed human/AI conversations, low–resource languages) and a deployment plan inside Erste's mobile banking app are discussed.

1 Introduction and motivation

Romance scams—increasingly executed by large language model (LLM) powered chatbots and known in their financialized form as "pig butchering"—are among the fastest growing categories of consumer fraud in Europe. A scammer cultivates an apparent romantic relationship with a victim over weeks or months, gradually isolates them from family and friends, and ultimately induces one or more large outbound transfers, often to a fraudulent "investment" platform.

Three properties make this problem a natural target for a bank.

1. **The victim is the bank's customer.** Unlike card fraud, where the bank can claw back unauthorized transactions, romance scams are *authorized push payment* fraud: the customer themselves initiates the transfer. Recovery rates are very low.

2. **The bank sees the moment of harm.** Whatever happens on Tinder, WhatsApp, or Facebook is invisible to the bank—but the transfer screen is not. A detector deployed there is the last reliable intervention point.

3. **The harm falls disproportionately on a loyal demographic.** Median romance–scam victims skew older, are more likely to use traditional retail banking, and suffer life–changing losses (a single victim losing six figures is routine).

We therefore frame the problem as follows. A customer initiating a transfer is invited (optionally) to paste the recent text conversation that motivated the transfer. Our detector consumes that conversation and returns

$$\hat{p}(\text{scam} \mid \text{conversation}) \in [0, 1],$$

together with a small set of human–readable red flags. A score above a calibrated threshold triggers an in–app warning before the transfer is executed.

The remainder of this report develops the mathematics behind $\hat{p}$. We deliberately favor interpretable, lightweight statistics over end–to–end deep classifiers: interpretability is non–negotiable for a banking deployment, and lightweight features can be computed on–device or on standard CPU infrastructure.

2 How the bot works

2.1 Generation: autoregressive language models

Modern scam chatbots are powered by autoregressive transformer language models. Given a token vocabulary $\mathcal{V}$ and a prefix $x_{<t} = (x_1, \ldots, x_{t-1})$, such a model defines a conditional distribution

$$p_\theta(x_t \mid x_{<t}) = \text{softmax}\big(W_o h_t\big), \qquad h_t = \text{Transformer}_\theta(x_{<t}),$$

trained by minimizing the cross–entropy loss

$$\mathcal{L}(\theta) = -\mathbb{E}_{x \sim \mathcal{D}} \left[\sum_{t=1}^{|x|} \log p_\theta(x_t \mid x_{<t}) \right]$$

on a large text corpus $\mathcal{D}$. Equivalently, training drives the model toward the maximum–likelihood estimator of $\mathcal{D}$'s distribution; the resulting samples concentrate on *high–probability* regions of p_θ. This concentration is the central fact we will exploit for detection.

A message is produced by sampling

$$x_t \sim \tilde{p}_\theta(\cdot \mid x_{<t}),$$

where $\tilde{p}_\theta$ is some sharpening of p_θ: temperature scaling $\tilde{p}_\theta(v) \propto p_\theta(v)^{1/\tau}$, top–$k$ truncation, or nucleus (top–p) truncation. In practice scammers use moderate temperatures ($\tau \in [0.7, 1.0]$) and top–p around 0.9.

2.2 The scam playbook

Operational scammers do not have the LLM improvise freely. They wrap it in a *system prompt* encoding a playbook with five canonical stages:

(a) **Opening / love bombing.** High–affect compliments, rapid escalation of intimacy.

(b) **Rapport building.** Daily small talk, shared "future" planning, mirroring the victim's interests.

(c) **Isolation.** Discouragement of consultation with family, framing the relationship as uniquely understood.

(d) **Hook / opportunity injection.** A casual mention of a profitable "investment," a sick relative, a customs fee, a stranded relative.

(e) **Urgency / extraction.** Time–limited request, emotional pressure, specific transfer instructions.

We will encode these as the hidden states of an HMM in Section 3.3.4.

2.3 The detectable signature

Two structural properties make scam chatbot conversations detectable.

Likelihood concentration. Because p_θ was trained by maximum likelihood and the scammer samples from it, the bot's messages typically have lower per–token cross–entropy under *any* large reference language model than do messages written by a non–native–speaking human under emotional load. This drives the perplexity, burstiness, curvature, and token–rank features of Section 3.2.

Scripted progression. The five–stage playbook induces a low–entropy trajectory through topic and sentiment space. Genuine conversations meander; scam conversations move on rails. This drives the conversational–dynamics and HMM features of Sections 3.3 and 3.3.4.

3 Mathematical analysis

3.1 Notation and pipeline

Let a conversation be an ordered sequence

$$C = \big((u_1, m_1, t_1), (u_2, m_2, t_2), \ldots, (u_T, m_T, t_T)\big),$$

where $u_i \in \{V, S\}$ marks the speaker (victim V or suspected scammer S), m_i is the message text, and t_i is the timestamp. Write $C_S = (m_i)_{u_i = S}$ for the subsequence of messages from the suspected scammer, and let $w(m) = (w_1, \ldots, w_{|m|})$ denote the token sequence of message m under a fixed reference tokenizer.

Our pipeline (Figure 1) extracts a feature vector $\phi(C) \in \mathbb{R}^d$ in three families:

$$\phi(C) = \big[\phi_{\text{LLM}}(C_S),\ \phi_{\text{dyn}}(C),\ \phi_{\text{sem}}(C_S)\big],$$

and feeds it to a binary classifier

$$\hat{p}(\text{scam} \mid C) = \sigma\big(w^\top \phi(C) + b\big), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Conversation $C \longrightarrow$ feature extractor $\phi \longrightarrow$ logistic regression $\sigma(w^\top \phi + b) \longrightarrow$ score $\hat{p} \in [0, 1]$ + red flags

Slika 1: Detection pipeline. All feature extractors are independently interpretable.

3.2 Token–level likelihood features

We use a publicly available pretrained model p_θ (GPT–2 medium in our implementation; see Section 3.8 for a discussion of this choice) as the reference for likelihood computations.

3.2.1 Perplexity

Definition 1 (Perplexity)**.** *For a token sequence $w = (w_1, \ldots, w_N)$,*

$$\mathrm{PP}_\theta(w) = \exp\left(-\frac{1}{N} \sum_{i=1}^{N} \log p_\theta(w_i \mid w_{<i}) \right).$$

Perplexity is the exponentiated average negative log–likelihood; it is the geometric mean of the inverse next–token probabilities. Lower perplexity means the reference model finds the sequence more expected.

Proposition 1 (Why scam chatbot text has lower expected perplexity)**.** *Let p_θ be a maximum–likelihood estimator of a text distribution $\mathcal{D}$, and let the scammer's bot sample (with mild sharpening) from a related distribution q_ϕ trained on a superset of $\mathcal{D}$. Then for a sample $w \sim q_\phi$,*

$$\mathbb{E}_{w \sim q_\phi}\big[-\log p_\theta(w)\big] \;\le\; \mathbb{E}_{w \sim \mathcal{H}}\big[-\log p_\theta(w)\big],$$

where $\mathcal{H}$ is the distribution of human messages of comparable length and topic, provided that $D_{\mathrm{KL}}(q_\phi \,\|\, p_\theta) < D_{\mathrm{KL}}(\mathcal{H} \,\|\, p_\theta)$.

The KL inequality holds empirically for human conversational text written under emotional strain, by non–native speakers, or both—which is overwhelmingly the case for real scam victims and their counterparties. We therefore include the mean per–message perplexity

$$\phi_1(C_S) = \frac{1}{|C_S|} \sum_{m \in C_S} \mathrm{PP}_\theta(w(m))$$

as a feature.

3.2.2 Burstiness

Human writing exhibits high *burstiness*: variance in per–sentence perplexity. AI–generated text is smoother. Define, for the j–th sentence s_j of C_S,

$$\pi_j = \log \mathrm{PP}_\theta\big(w(s_j)\big),$$

and let

$$\phi_2(C_S) = \frac{\sigma_\pi - \mu_\pi}{\sigma_\pi + \mu_\pi} \in [-1, 1], \qquad \mu_\pi = \frac{1}{J}\sum_j \pi_j,\ \sigma_\pi^2 = \frac{1}{J}\sum_j (\pi_j - \mu_\pi)^2.$$

This is the Goh–Barabási burstiness coefficient applied to log–perplexity. Higher values indicate more human–like variability.

3.2.3 Local curvature (DetectGPT–style)

The key insight of Mitchell et al. [1] is that AI–generated text sits at *local maxima* of $\log p_\theta$. Concretely, let $q(\cdot \mid x)$ be a small text–perturbation distribution (we use a T5 mask–and–fill model, masking 15% of tokens). Define the curvature statistic

$$d(x) = \log p_\theta(x) \; - \; \mathbb{E}_{\tilde{x}\sim q(\cdot|x)}\big[\log p_\theta(\tilde{x})\big].$$

A second–order Taylor expansion of $\log p_\theta$ around x shows that

$$d(x) \approx -\tfrac{1}{2}\,\mathrm{tr}\big(\Sigma_q \nabla^2 \log p_\theta(x)\big) + O(\|\Sigma_q\|^{3/2}),$$

i.e. $d(x)$ estimates (a scaled trace of) the Hessian of $\log p_\theta$ at x. AI samples, being local maxima, have $\nabla^2 \log p_\theta \prec 0$ and hence $d(x) > 0$ with margin; human samples are not generally at local maxima and exhibit $d(x) \approx 0$.

We estimate $d(x)$ by Monte Carlo with K perturbations and normalize by an empirical standard deviation:

$$\hat{d}(x) \;=\; \frac{\log p_\theta(x) - \frac{1}{K}\sum_{k=1}^{K} \log p_\theta(\tilde{x}_k)}{\hat{\sigma}_x}, \qquad \hat{\sigma}_x^2 = \frac{1}{K-1}\sum_{k=1}^{K}\Big(\log p_\theta(\tilde{x}_k) - \tfrac{1}{K}\sum_{k'} \log p_\theta(\tilde{x}_{k'})\Big)^2.$$

We aggregate per message and average: $\phi_3(C_S) = \frac{1}{|C_S|}\sum_m \hat{d}(m)$.

3.2.4 Token–rank distribution (GLTR features)

For each token w_i of a message, define its rank in the reference distribution:

$$r_i = \big|\{v \in \mathcal{V} : p_\theta(v \mid w_{<i}) \geq p_\theta(w_i \mid w_{<i})\}\big|.$$

AI–generated text disproportionately uses tokens with $r_i \leq 10$. Two features:

$$\phi_4(C_S) = \frac{\#\{i : r_i \leq 10\}}{N}, \qquad \phi_5(C_S) = \frac{\#\{i : r_i \leq 100\}}{N}.$$

3.2.5 Classical stylometric features

We complement likelihood features with three classical stylometric statistics computed on C_S as a whole.

Type–token ratio measures lexical diversity:

$$\mathrm{TTR}(C_S) = \frac{V(N)}{N}, \qquad \phi_6 = \mathrm{MATTR}_W(C_S) = \frac{1}{N-W+1}\sum_{i=1}^{N-W+1}\frac{V_i(W)}{W},$$

the moving–average TTR over windows of size $W = 100$ (length–robust).

Yule's K is a length–independent vocabulary–richness statistic:

$$\phi_7 = K = 10^4 \cdot \frac{\sum_{i\geq 1} i^2\, V(i,N) - N}{N^2},$$

where $V(i,N)$ is the number of types occurring exactly i times. Lower K means more diverse vocabulary; scam scripts that recycle a small repertoire of pet names and pressure phrases tend toward higher K.

Zipf–exponent deviation. Empirical word frequencies $f(r)$ ranked by r are well modeled by Zipf's law $f(r) \propto r^{-s}$ with $s \approx 1$ for natural English. Estimate $\hat{s}$ by ordinary least squares on $\log f = -s \log r + c$ restricted to ranks $r \in [10, 1000]$, and only when at least 150 tokens are available (Zipf is an asymptotic property; below this threshold the OLS fit is dominated by noise and the feature returns NaN, which is then imputed at the classifier stage). Define

$$\phi_8 = |\hat{s} - 1|.$$

3.3 Conversational–dynamics features

3.3.1 Asymmetry

A scammer working a script produces longer, more polished messages than an emotionally invested victim writing replies. Let

$$\ell(m) = \text{token count of } m, \qquad \bar{\ell}_S = \text{mean}_{u_i=S}\ell(m_i), \quad \bar{\ell}_V = \text{mean}_{u_i=V}\ell(m_i).$$

Define

$$\phi_9 = \log \frac{\bar{\ell}_S + 1}{\bar{\ell}_V + 1}.$$

A second asymmetry feature uses response–time variance: scammers' inter–message intervals show shift–patterns (operators handling multiple targets) detectable as bimodality. Let $\Delta_i = t_i - t_{i-1}$ for $u_i = S$; we compute

$$\phi_{10} = \text{Bimodality-Coefficient}(\{\Delta_i\}) = \frac{\gamma_1^2 + 1}{\gamma_2 + 3(n-1)^2/((n-2)(n-3))},$$

with γ_1, γ_2 the sample skewness and excess kurtosis.

3.3.2 Sentiment trajectory and CUSUM change–point detection

Each scammer message m_i is scored for sentiment $s_i \in [-1, 1]$ using a multilingual transformer–based sentiment classifier (`cardiffnlp/twitter-xlm-roberta-base-sentiment`). The romance–scam trajectory follows a characteristic pattern: a long high–positive plateau (love bombing, rapport), followed by a sharp negative deflection (urgency, emotional pressure) coincident with the financial ask.

We capture this with a one–sided CUSUM test against the null of a stationary positive baseline $\mu_0 = \text{median}(s_{1:\lfloor T/2 \rfloor})$:

$$G_0 = 0, \qquad G_t = \max\bigl(0,\ G_{t-1} + (\mu_0 - s_t - \kappa)\bigr),$$

with reference value $\kappa = 0.25$. Under the null G_t is bounded; under a negative shift it grows linearly. Features:

$$\phi_{11} = \max_t G_t, \qquad \phi_{12} = \text{slope}_{\text{OLS}}(s_t \text{ on } t).$$

3.3.3 Topic–shift via Kullback–Leibler divergence

A scam conversation injects financial/urgency vocabulary late. Partition C_S into an early window W_E (first half) and a late window W_L (second half), yielding token–count maps n_E and n_L. Let $\mathcal{V}'(C_S) = \{v : n_E(v) > 0 \text{ or } n_L(v) > 0\}$ be the observed–token vocabulary, and let $\mathcal{F}$ be a curated lexicon of $\sim$200 financial / urgency / scam–indicative terms (full list in `data/scam_lexicon.txt`).

For ϕ_{13} we apply Laplace smoothing with $\alpha = 0.5$ to avoid $\log 0$ in the KL sum:

$$\tilde{p}_E(v) = \frac{n_E(v) + \alpha}{\sum_{v'} n_E(v') + \alpha\,|\mathcal{V}'|}, \qquad \tilde{p}_L(v) = \frac{n_L(v) + \alpha}{\sum_{v'} n_L(v') + \alpha\,|\mathcal{V}'|},$$

$$\phi_{13} = D_{\text{KL}}\bigl(\tilde{p}_L \,\|\, \tilde{p}_E\bigr) = \sum_{v \in \mathcal{V}'} \tilde{p}_L(v) \log \frac{\tilde{p}_L(v)}{\tilde{p}_E(v)}.$$

For ϕ_{14} we use the empirical (unsmoothed) late distribution against the finance lexicon directly:

$$\phi_{14} = \frac{\#\{w \in C_S^{\text{late}} : w \in \mathcal{F}\}}{|C_S^{\text{late}}|}.$$

Smoothing is deliberately omitted from ϕ_{14}. With $|\mathcal{F}| \approx 200$ and short conversations, an additive constant per lexicon item dominates the actual usage signal and can invert the metric, mapping benign casual chats to artificially high finance mass.

3.3.4 HMM over scam stages

We model the playbook of Section 2.2 as a hidden Markov model $\lambda = (\pi, A, B)$ with five hidden states $\mathcal{S} = \{\text{Open}, \text{Rapport}, \text{Isolate}, \text{Hook}, \text{Urgency}\}$.

The emission distribution in each state is categorical over a discrete observation alphabet $\mathcal{O}$. We construct $\mathcal{O}$ by binning a 5-dimensional per-message feature vector

$$\psi(m) = \big(\text{sentiment bin}, \text{length bin}, \text{finance-lexicon hit}, \text{question-rate bin}, \text{position bin}\big)$$

with bin counts $(3, 3, 2, 3, 3)$, giving $|\mathcal{O}| = 162$ symbols. Each message is mapped to a single symbol $o(m) \in \{0, \ldots, 161\}$ via mixed-radix encoding. The sentiment bin is obtained from a lightweight positive/negative wordlist heuristic (cheaper than running the transformer sentiment model used for ϕ_{11}, ϕ_{12} at every HMM observation); length and question-rate bins use empirical thresholds; the position bin records whether m falls in the first, middle, or last third of C_S.

We restrict A to be upper triangular with self-loops; scam conversations rarely retreat from Urgency back toward Rapport. The initial distribution π is concentrated on Open. Parameters (π, A, B) are fit by Baum–Welch on the labeled scam training corpus, with $|\mathcal{S}| \cdot |\mathcal{O}| = 810$ emission parameters total.

Given a fitted scam HMM λ_S and a fitted "normal–conversation" HMM λ_N (same topology, fit on benign dialogue corpora), we score a new conversation by the log–likelihood ratio

$$\phi_{15}(C_S) = \log P(C_S \mid \lambda_S) - \log P(C_S \mid \lambda_N),$$

computed via the forward algorithm:

$$\alpha_t(j) = \Big[\sum_i \alpha_{t-1}(i)\, A_{ij}\Big] B_{j,\, o(m_t)}, \qquad P(C_S \mid \lambda) = \sum_j \alpha_T(j).$$

We additionally extract, via the Viterbi algorithm, the maximum–likelihood stage path; the existence of an Urgency state in the path is a binary feature ϕ_{16}, and is used as a human–readable red flag.

3.4 Semantic–similarity features

Real scam operations reuse scripts at scale. We exploit this by embedding messages in a multilingual sentence–transformer space $\varphi : \text{text} \to \mathbb{R}^{384}$ (we use `paraphrase-multilingual-MiniLM-L12-v2`) and comparing against a corpus $\mathcal{T}$ of confirmed scam messages. In our implementation $\mathcal{T}$ is bootstrapped from a seed of 25 hand–curated scam–letter templates and grown by self–extraction: every scammer–side message of 8–80 words from the training corpus is added to $\mathcal{T}$, giving $|\mathcal{T}|$ on the order of 10^3 in our final corpus. In production $\mathcal{T}$ would be regularly enriched from internal fraud–report archives and public sources (e.g. scamletters.com), with $|\mathcal{T}|$ scaling into the tens of thousands.

For each scammer message $m \in C_S$ define

$$\rho(m) = \max_{t \in \mathcal{T}} \frac{\langle \varphi(m), \varphi(t) \rangle}{\|\varphi(m)\|\, \|\varphi(t)\|},$$

and aggregate:

$$\phi_{17} = \max_{m \in C_S} \rho(m), \qquad \phi_{18} = \text{mean}_{m \in C_S} \rho(m), \qquad \phi_{19} = \frac{\#\{m : \rho(m) > 0.75\}}{|C_S|}.$$

ϕ_{17} captures the worst–case (most template–like) message; ϕ_{19} captures pervasive scripting.

3.5 Feature fusion and classification

The full feature vector is $\phi(C) = (\phi_1, \ldots, \phi_{19}) \in \mathbb{R}^{19}$. We standardize each component to zero mean and unit variance using training–set statistics (with median imputation for NaN entries that arise when a feature's preconditions are not met, e.g. Zipf's law on short messages) and fit an ℓ_2–regularized logistic regression by minimizing

$$\mathcal{J}(w, b) = -\frac{1}{n} \sum_{k=1}^{n} \Big[y_k \log \sigma(w^\top \phi_k + b) + (1 - y_k) \log\big(1 - \sigma(w^\top \phi_k + b)\big) \Big] + \frac{\lambda}{2} \|w\|_2^2,$$

with λ chosen by 5–fold cross–validation on the training set.

The choice of logistic regression over a deep classifier is deliberate. Coefficients w_j are directly interpretable as log–odds contributions: a banking auditor can read off "feature ϕ_{15} (HMM stage–path) contributed +1.4 log–odds, feature ϕ_{17} (semantic template match) contributed +0.9 log–odds" and explain a decision to a customer. This interpretability is a regulatory requirement (GDPR Article 22; EU AI Act high–risk obligations) for an automated decision system deployed by a bank.

3.6 Evaluation

We evaluate on a held–out test set of conversations using

- **ROC AUC** as the primary discrimination metric;
- **Precision at fixed recall** (PR curve) since false positives carry real cost—warning a customer about a genuine partner is harmful;
- **Expected calibration error** (ECE) over 10 bins, since the score $\hat{p}$ is meant to drive a human–in–the–loop warning, not an automatic block;
- **Feature–family ablation**: AUC change when each of the five feature families $\{\phi_{\text{LLM}}, \phi_{\text{style}}, \phi_{\text{dyn}}, \phi_{\text{H}}$ is removed from the full model.

Directional pre–validation. Before fitting the classifier, we verified on a small held–out fixture set that each feature separates scam from benign in the direction predicted by the theory (e.g. ϕ_{14} larger for scam, ϕ_2 smaller). This catches sign errors and smoothing artifacts that an aggregate AUC would hide. The verification surfaced one such bug in an early version of ϕ_{14} where Laplace smoothing over an oversized lexicon inverted the metric on short conversations; the corrected definition appears in Section 3.3.3. Two features (ϕ_2 burstiness, ϕ_{13} topic–KL) showed insufficient separation at small sample sizes ($n < 10$ per class); we document this as a known limitation and rely on the classifier's ℓ_2–regularized fusion with stronger signals ($\phi_9, \phi_{14}, \phi_{17}$) for these to contribute fractional information.

Reported numbers are in Table 1 (to be filled in from the implementation).

Tablica 1: Detector performance, 5–fold cross–validated. Each ablation row removes one feature family from the full model. (Numbers to be filled in from the implementation.)

Configuration	AUC	Precision@90% recall	F1	ECE
Full model (all 19 features)	—	—	—	—
$-\phi_{\text{LLM}}$ (no $\phi_{1\text{–}5}$)	—	—	—	—
$-\phi_{\text{style}}$ (no $\phi_{6\text{–}8}$)	—	—	—	—
$-\phi_{\text{dyn}}$ (no $\phi_{9\text{–}14}$)	—	—	—	—
$-\phi_{\text{HMM}}$ (no $\phi_{15,16}$)	—	—	—	—
$-\phi_{\text{sem}}$ (no $\phi_{17\text{–}19}$)	—	—	—	—
DetectGPT score alone (baseline)	—	—	—	—
Perplexity threshold (baseline)	—	—	—	—

3.7 Design choices and trade-offs

Several decisions in the pipeline above represent deliberate trade-offs rather than the only mathematically valid construction. We document them explicitly, both because the choices affect deployment behavior and because the reasoning extends naturally to future engineering iterations.

Reference language model: GPT-2 medium (355M). The DetectGPT-style features (ϕ_3) and the perplexity / token-rank features ($\phi_1, \phi_2, \phi_4, \phi_5$) require a publicly available reference language model. We chose GPT-2 medium over both smaller (GPT-2 base, 124M) and larger (Llama-3-8B, Mistral-7B) alternatives for three reasons: (i) it runs on CPU in seconds per conversation, which preserves the on-device deployment story; (ii) its tokenizer and Hugging Face integration are exceptionally stable; (iii) cross-model DetectGPT [1] retains discriminative power even when the reference is substantially smaller than the generator, because likelihood landscapes of internet-trained transformers are highly correlated. Section 3.8 discusses the expected degradation against frontier generators and the one-line upgrade path to a 7–8B-parameter reference.

Logistic regression over deep classifier. A small neural network or gradient-boosted ensemble would likely improve raw AUC by a few points. We rejected this because (i) GDPR Article 22 and EU AI Act high-risk obligations require per-decision explanations that a 19-D logistic regression provides directly through coefficient inspection, while a neural ensemble requires post-hoc methods (SHAP, LIME) that auditors view as approximations; (ii) at the training-set sizes available without invoking expensive synthetic generation ($n \sim 400$), a 19-parameter model with ℓ_2 regularization is sample-efficient where a deep model is not.

Fixed 19-D feature vector instead of learned representations. We deliberately compute features rather than learn them. Each ϕ_i has a mathematical definition, a peer-reviewed precedent, and an interpretable direction (e.g. ϕ_{14} is the proportion of late scammer-side tokens drawn from the financial lexicon). This is the single most important property for a banking deployment: every alert can be decomposed into "which features fired, by how much."

Two HMMs (λ_S, λ_N) and their log-likelihood ratio. Rather than train a single discriminative HMM, we fit independent generative models for scam and benign conversations

on their respective training subsets. The log-likelihood ratio $\log P(C_S \mid \lambda_S) - \log P(C_S \mid \lambda_N)$ is the Neyman–Pearson optimal statistic for the binary hypothesis test under the modeling assumptions and reuses the standard forward algorithm without modification.

CategoricalHMM with 162-symbol mixed-radix observations. A continuous-emission HMM (e.g. GaussianHMM over $\psi(m) \in \mathbb{R}^5$) would have $|\mathcal{S}| \cdot (5 + 5 \cdot 6/2) = 100$ emission parameters, comparable to our 810. We chose the discrete form because Baum–Welch on a categorical emission with bounded support has substantially better small-sample behavior, particularly when the training corpus has $n \sim 200$ scam conversations.

Lightweight sentiment heuristic inside the HMM. The HMM emission feature includes a sentiment bin. We compute it from a small positive/negative wordlist rather than calling the multilingual transformer (ϕ_{11}, ϕ_{12}). The transformer model would add $\sim$1 second per HMM observation; the wordlist costs microseconds. This is the dominant choice keeping training time linear in corpus size.

Median imputation for missing features. Any feature whose preconditions fail (e.g. Zipf's law on a conversation of < 150 scammer-side tokens) returns NaN. We impute with the training- set median rather than zero or mean, because (i) zero conflates "missing" with "small," which is meaningful for several features, and (ii) the median is robust to the long-tailed distributions several features exhibit (e.g. ϕ_7 Yule's K).

Semantic features by cosine similarity, not fine-tuning. Given a corpus $\mathcal{T}$ of scam templates, one alternative is to fine-tune a classifier head on $\{\varphi(t) : t \in \mathcal{T}\}$. We use unmodified cosine similarity instead because it is fully zero-shot: new templates can be added to $\mathcal{T}$ at any time without any retraining, which matches the realistic operational scenario of a bank fraud-intelligence team continuously enriching the template database.

Retention of weak features (ϕ_2, ϕ_{13}). Directional pre-validation (Section **??**) showed that two features have insufficient separation at very small sample sizes. We keep them in the vector for two reasons: their theoretical mechanism is sound (burstiness is a documented LLM signature; topic-shift KL is a mainstream technique), and ℓ_2-regularized logistic regression on a held-out development set is the principled way to weight an uncertain feature, including potentially down-weighting it to near-zero. "Remove it because it doesn't fire" would be premature; "let the classifier decide" is the correct discipline.

3.8 Strengths and limitations

Strengths. The detector is fully interpretable, runs on CPU in under a few seconds per conversation, requires no training of large models (we only fine–tune a 19–dimensional logistic regression), works in multiple languages via the multilingual sentence transformer and tokenizer, and gracefully degrades if any one feature family is unavailable (e.g. no timestamps, no trained HMM, no GPU access).

Limitations.

1. *Paraphrase attack.* A scammer who routes LLM output through a second "humanizing" paraphraser can defeat the LLM–likelihood features. The conversational–dynamics and semantic features remain effective in this regime; the feature–ablation analysis quantifies this.

2. *Mixed conversations.* If a human scammer occasionally interleaves their own writing with LLM output, per–message scores become noisy; aggregation over the conversation mitigates this only partially.

3. *Low–resource languages.* Reference perplexity and sentiment models are weaker outside English/Spanish/German/French/Mandarin, degrading the LLM and sentiment–trajectory features. Conversational and semantic features remain language–agnostic.

4. *Reference–model capacity gap.* Our reference model p_θ is GPT–2 medium (355M parameters, public weights), substantially smaller than the frontier models a scammer is likely to use (GPT–4–class, Claude–class, $\sim 10^{11}$–10^{12} parameters). DetectGPT–style curvature signals weaken as the gap widens: a frontier sample is also a local maximum under its own distribution, but its local maxima under our smaller reference need not coincide. Using a larger open–weight reference (Llama–3–8B, Mistral–7B) closes some of this gap and is a straightforward upgrade.

5. *Adversarial drift.* As detection methods become known, scammers will adapt prompts (introducing intentional typos, simulated bursts of emotion, randomized message lengths). Periodic retraining and feature engineering are mandatory; see Section 5.

6. *Calibration depends on prior.* The score $\hat{p}$ is meaningful only under the training prior $P(\text{scam})$; in deployment the bank should re–calibrate using empirical base rates from its own customer population.

4 Implementation

The detector is implemented in Python 3.13. The repository (delivered alongside this report) is organized as:

```
bot-detector/
  data/              # scam + benign conversation corpora (JSONL)
                     # plus scam_lexicon.txt, scam_templates.jsonl
  src/
    features/
      perplexity.py      # phi_1, phi_2 (PP, burstiness)
      detect_gpt.py      # phi_3 (curvature)
      token_rank.py      # phi_4, phi_5 (GLTR)
      stylometry.py      # phi_6, phi_7, phi_8
      asymmetry.py       # phi_9, phi_10
      sentiment.py       # phi_11, phi_12 (CUSUM)
      topic_shift.py     # phi_13, phi_14 (KL)
      hmm_stages.py      # phi_15, phi_16
      semantic.py        # phi_17, phi_18, phi_19
    pipeline.py      # assembles phi(C)
    train.py         # fits logistic regression + HMMs, saves model
    detect.py        # CLI entry point
    evaluate.py      # ROC/PR/ECE/ablations
    data_collection.py  # reddit, huggingface, synth, templates
        subcommands
    scrape_selenium.py  # configurable scraper for public scam archives
  tests/conversations/   # example inputs (.json)
  models/                # trained classifier + HMMs
  compare_features.py    # directional sanity check
  requirements.txt
  README.md
  DATA_COLLECTION.md     # data sources, scraping, ethics
```

```
  MATH_REFERENCE.md      # one-page math summary
```

Data

Training and evaluation use the `BothBosu/Scammer-Conversation` dataset from Hugging Face: 1 000 synthetic two–speaker phone–dialogue conversations with binary scam/non–scam labels, generated by an LLM following social–engineering playbooks (financial impersonation, refund, reward, technical support, SSN). For our experiments we use 200 scam and 200 benign conversations, with 5–fold cross–validation. The `scrape_selenium.py` module and a Reddit collector in `data_collection.py` provide paths to expand to real–world data; both are documented in the auxiliary `DATA_COLLECTION.md`.

4.1 Inference: how a user runs the detector

A conversation is supplied as a JSON file with one object per message:

```
{
  "label": 1,
  "source": "...",
  "messages": [
    {"speaker": "S", "text": "Good morning my dear, I missed you all night
        .",
     "timestamp": "2026-04-12T07:31:00Z"},
    {"speaker": "V", "text": "morning :)", "timestamp": "..."}
  ]
}
```

The user runs:

```
$ python -m src.detect --input tests/conversations/scam_01.json

Romance-scam probability:  0.93
Top red flags:
  - Semantic match to known scam template (max cos = 0.86)
  - HMM Viterbi path enters Urgency stage
  - Late-conversation financial lexicon mass = 0.18
  - Topic-shift KL divergence = 1.7
  - DetectGPT curvature z-score = 3.1
  - CUSUM sentiment-shift statistic = 2.4
```

4.2 Key algorithm: full pipeline

Algorithm 1 Romance–scam detection pipeline

Require: Conversation C, reference LM p_θ, perturbation model q, embedding φ, scam template set $\mathcal{T}$, fitted HMMs λ_S, λ_N, classifier (w, b)

$C_S \leftarrow \{m_i \in C : u_i = S\}$
Compute ϕ_1, ϕ_2 by running p_θ on each $m \in C_S$
Compute ϕ_3 by sampling $K = 10$ perturbations per message from q and evaluating p_θ
Compute ϕ_4, ϕ_5 from per–token ranks
Compute ϕ_6, ϕ_7, ϕ_8 from token / type counts of C_S
Compute ϕ_9, ϕ_{10} from message lengths and timestamps
Compute ϕ_{11}, ϕ_{12} from sentiment scores s_t
Compute ϕ_{13}, ϕ_{14} from early/late word distributions
Compute ϕ_{15} via forward algorithm on λ_S and λ_N; ϕ_{16} via Viterbi
Compute $\phi_{17}, \phi_{18}, \phi_{19}$ via embedding similarity against $\mathcal{T}$
$\phi \leftarrow$ median–impute and standardize $(\phi_1, \ldots, \phi_{19})$ with training statistics
return $\sigma(w^\top \phi + b)$ and the set of features above a per–feature alert threshold

4.3 Test examples

The directory `tests/conversations/` contains six labeled example conversations: three synthetic scam dialogues modeled on documented romance–scam playbooks (pig–butchering crypto, military–deployment persona, widower with hospitalized child), and three synthetic benign conversations (first–date planning, long–distance partners, friends catching up). Training data is drawn separately from the `BothBosu/Scammer-Conversation` Hugging Face corpus, supplemented by templates self–extracted from that corpus. The README documents how to add new test cases.

5 Further development

5.1 Deployment inside the bank

The natural integration point is the outbound–transfer screen of the Erste mobile banking app. When the customer initiates a transfer above a configurable threshold (e.g. EUR 500) to a new or low–frequency beneficiary, the app presents an optional "share recent conversation" flow. If the score $\hat{p}$ exceeds a calibrated threshold, the customer is shown an inline warning with the top three red flags (e.g. "This conversation contains a pattern matching known investment–scam scripts"), a 24–hour cool–down option, and a link to a dedicated fraud–prevention specialist.

The feature pipeline is light enough that classifier inference itself runs on–device; the underlying reference models (GPT–2 medium, T5–small, XLM–RoBERTa, MiniLM) total approximately 2 GB and would either need to be quantized for on–device deployment or moved to a privacy–preserving inference gateway with no message logging. The latter reduces GDPR exposure substantially compared to a naive "upload your chat" design.

5.2 Extensions

Browser / messaging–app extension. The same feature pipeline can be wrapped as a browser extension for web–based dating platforms or as a Signal/Telegram plug–in, scoring

conversations passively and warning the user before any transfer is initiated.

Multimodality. The Erste application can additionally analyze the counterparty's profile photo with a frequency–domain deepfake detector (radial 2D–FFT power spectrum, Benford–on–DCT). Photo and text features can be fused into the same logistic regression. We sketched this extension but did not implement it for the present report.

Continual learning. Each warned transfer that is subsequently confirmed as scam (e.g. via a chargeback request, police report, or victim disclosure) provides a positive label; each warned transfer that proceeds without complaint provides (with caution) a negative label. A monthly retraining loop, with careful handling of label noise and concept drift, keeps the detector current.

Multilingual expansion. The stylometric, conversational, and HMM features are largely language–independent. The LLM–likelihood features require a per–language reference model; multilingual models like XGLM and BLOOM provide adequate coverage for Erste's footprint (Croatian, Serbian, Hungarian, Czech, Romanian, Slovak).

5.3 How long can the detector remain effective?

The token–level features (perplexity, burstiness, curvature, token rank) have a known shelf life: as commercial LLMs become better at imitating informal, error–prone human writing, the likelihood gap shrinks. We estimate a 12–24 month useful horizon for these features without active counter–measures (e.g. targeted retraining against the latest models, ensemble of reference models, upgrade of the reference model from GPT–2 medium to a 7–8B–parameter open–weight model).

The conversational–dynamics, HMM, and semantic–template features have a much longer horizon. The romance–scam *playbook* is dictated by the underlying social engineering, not by the choice of generation technology; even a perfect, human–indistinguishable LLM still has to escalate, isolate, and request. As long as scammers are economically rational, the HMM stage structure persists.

6 References

Literatura

[1] E. Mitchell, Y. Lee, A. Khazatsky, C. Manning, C. Finn. *DetectGPT: Zero–Shot Machine–Generated Text Detection using Probability Curvature.* ICML, 2023.

[2] A. Hans et al. *Spotting LLMs With Binoculars: Zero–Shot Detection of Machine–Generated Text.* ICML, 2024.

[3] S. Gehrmann, H. Strobelt, A. Rush. *GLTR: Statistical Detection and Visualization of Generated Text.* ACL, 2019.

[4] J. Kirchenbauer et al. *A Watermark for Large Language Models.* ICML, 2023.

[5] F. J. Tweedie, R. H. Baayen. *How Variable May a Constant Be? Measures of Lexical Richness in Perspective.* Computers and the Humanities, 32(5), 1998.

[6] K.–I. Goh, A.–L. Barabási. *Burstiness and memory in complex systems.* EPL, 81(4), 2008.

[7] L. R. Rabiner. *A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition.* Proceedings of the IEEE, 77(2), 1989.

[8] N. Reimers, I. Gurevych. *Sentence–BERT: Sentence Embeddings using Siamese BERT–Networks.* EMNLP, 2019.

[9] J. M. Whittaker, M. T. Button. *Understanding pet ownership in pig–butchering style romance scams.* [Insert appropriate empirical reference for romance–scam victim demographics and loss statistics.]

[10] U.S. Federal Trade Commission, Consumer Sentinel Network Data Book 2023. [Latest FTC figures on romance–scam losses — verify and update with EU figures from Europol IOCTA for the final submission.]

[11] BothBosu. *Scammer-Conversation: Synthetic multi-turn scam and non-scam dialogue dataset.* Hugging Face, 2024. `https://huggingface.co/datasets/BothBosu/Scammer-Conversation`